

Exercício de Fixação — Subprogramas

Página / Slide 23

Aluno: Rodrigo Del Padre

RA: 24022092-2

Curso: Engenharia de Software (6º Semestre)

Turma: B

Disciplina: Paradigmas de Programação

Referência: R. Sebesta, 11ª ed., Caps. 9 e 10

Trecho 5 — Rust

Código analisado:

```
fn dobra(v: Vec<i32>) -> Vec<i32> {
    v.iter().map(|x| x * 2).collect()
}

let v = vec![1, 2, 3];
let d = dobra(v);
println!("{:?} {:?}", v, d);
```

(a) Qual é a saída?

Erro em Tempo de Compilação: error[E0382]: borrow of moved value: `v`

(b) Qual conceito da aula explica?

Conceito Central: Sistema de Posse (*Ownership*), Semântica de Movimentação (*Move Semantics*) e o *Borrow Checker* (Sebesta, 11ª ed., §9.5.4, p. 384; Slides 12, 13 e Síntese no Slide 24).

- **Posse Padrão por Valor (*Move*):** Em Rust, estruturas alocadas dinamicamente no *heap* (como `Vec<i32>`) não implementam a trait `Copy`. Ao passar `v` para a função `dobra(v)`, a posse do recurso é **movida** para o parâmetro formal da função.
- **Invalidação da Variável:** O identificador `v` no chamador perde a propriedade do vetor e torna-se imediatamente inválido.
- **Intervenção do Borrow Checker:** Ao tentar ler `v` em `println!("{:?} {:?}", v, d);`, o compilador bloqueia a compilação por empréstimo de valor movido.
- **Solução Idiomática:** Para emprestar sem transferir posse, utiliza-se referência imutável: `fn dobra(v: &[i32]) -> Vec<i32>` e chamada `dobra(&v)`.

Trecho 6 — Python

Código analisado:

```
total = 0

def adiciona(x):
    total = total + x
    return total
```

```
print(adiciona(5))
```

(a) Qual é a saída?

Erro em Tempo de Execução: `UnboundLocalError: cannot access local variable 'total' where it is not associated with a value`

(b) Qual conceito da aula explica?

Conceito Central: Resolução Estática de Escopo Local vs. Global (Regra LEGB e Vinculação de Nomes) (Sebesta, 11ª ed., §9.4 e §9.5.4; Slides 7, 12 e Síntese no Slide 24).

- **Classificação Léxica na Compilação:** Em Python, qualquer atribuição (`total = ...`) dentro do corpo de uma função marca aquele identificador como **variável local** para todo o subprograma, em tempo de compilação para *bytecode*.
- **Tentativa de Leitura Não-Vinculada:** Ao avaliar a expressão da direita `total + x`, o interpretador busca `total` apenas no registro de ativação local. Como ela ainda não recebeu atribuição nessa chamada, o acesso falha.
- **Isolamento do Escopo Global:** O interpretador **não** recorre à variável externa `total = 0` do módulo global porque a variável foi explicitamente classificada como local.
- **Solução:** Se necessária a mutação da variável global, declara-se `global total` dentro da função, ou idealmente passa-se o valor acumulado como parâmetro puro.